

Executor Framework

Thread
↳ manual

→ Memory Allocation

Thread t1 = new Thread(¹);

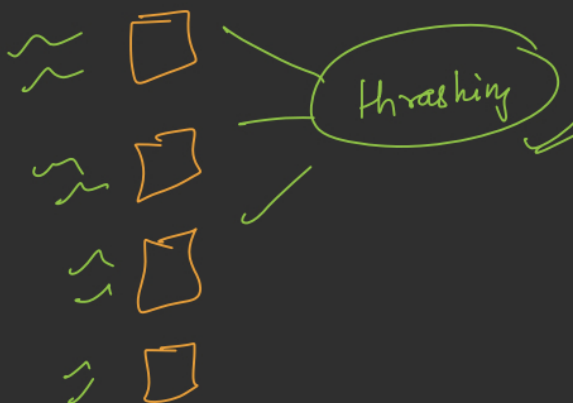
t1.start()

512 KB - 1 MB

1000 → 512 MB - 1 GB

→ OS Scheduling

↳ 1000 Threads



→ Thread Creation Time

↳ OS

→ (i) time use.

[Bad Approach]



Executor Framework

Threads

① Task

② How to do?

↳ Thread
↳ Runnable

Task

Executor Framework

Thread Pool

Thread Pool

↳ Reuse

Task1
Task2

Task3

Task4

Task5

Task6

Task7

Task8

T8

T9

T10

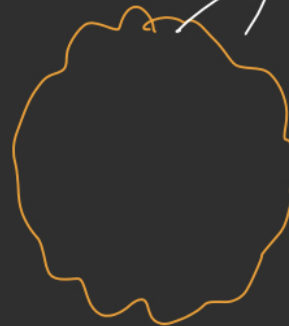
T11

T11 T10 T9 T8

Queue

Runnable

workers



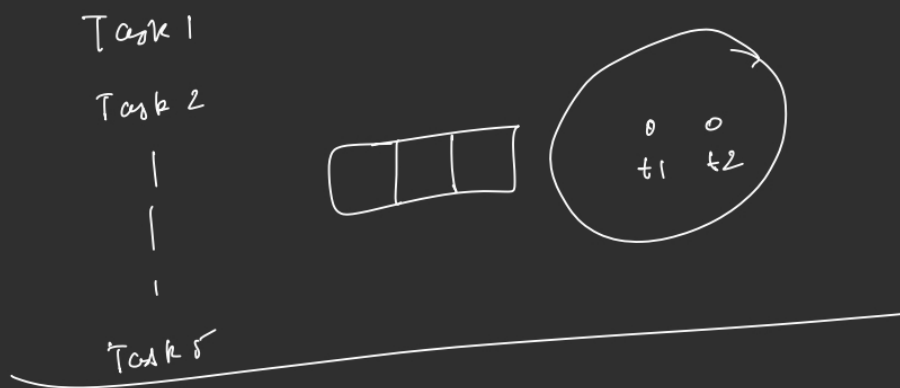
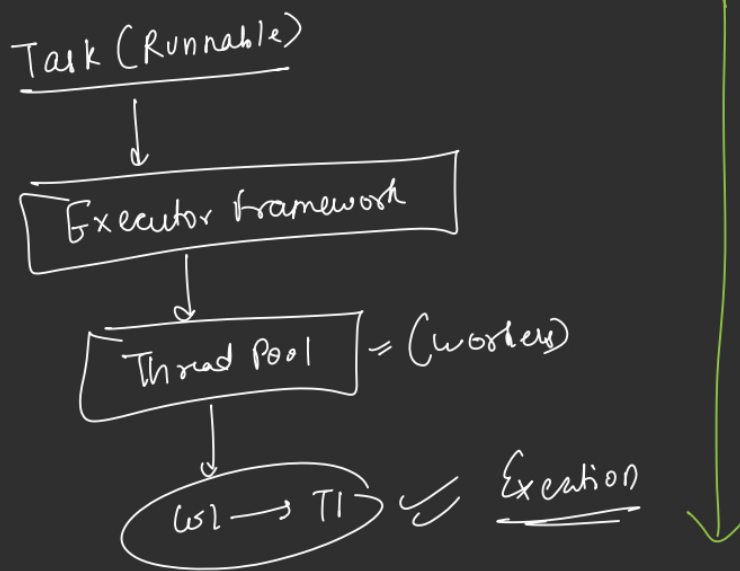
~~Task1~~ → b1

~~Task2~~ → b2

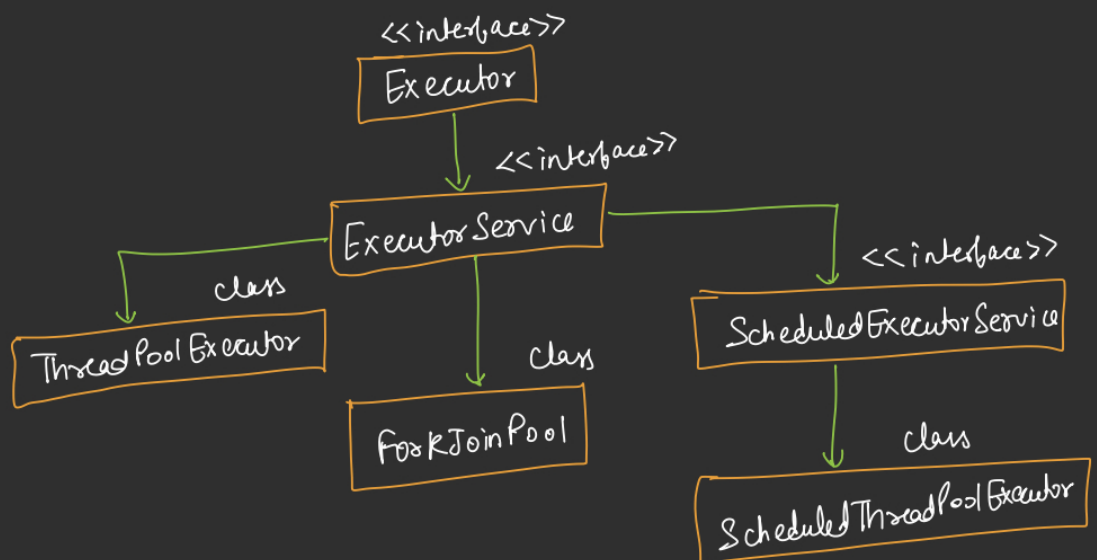
Task3 → i3

Task4 → +4

Task5 → +5



→ web development



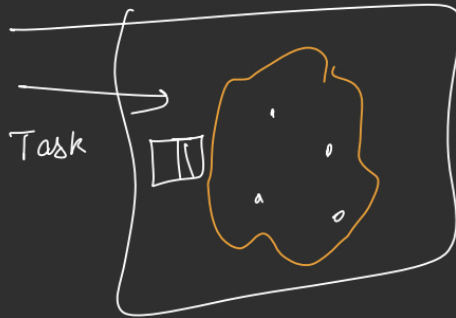
```
interface Executor {
    void execute(Runnable r);
}
```

Executor Service

execute();

submit();

executor.execute(-)



executor.execute() $\rightarrow$ `System.out.println("Hello");`

executor.submit() $\rightarrow$ `( ) \rightarrow 10 + 20;`

Callable

call()

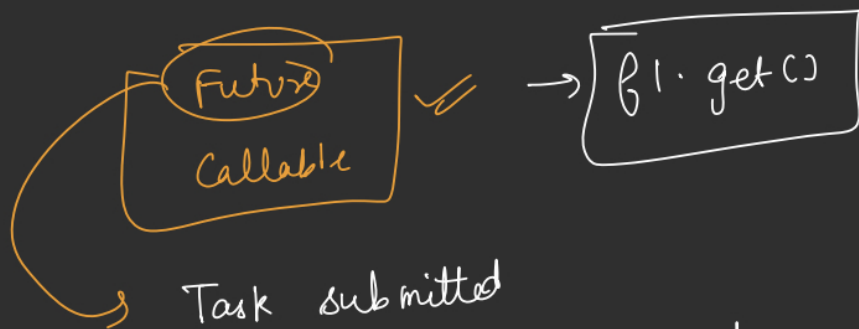
Return type

Runnable

run()

No return type

Future<Int> f1 = executor.submit (() -> 10);



Task submitted

Task run background thread.

Result stored in Future

f1.get() -> output

execute() -> Runnable

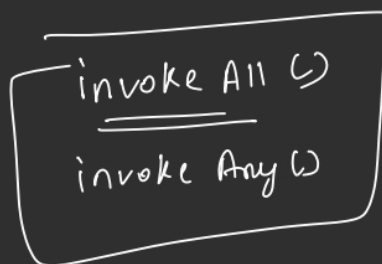
submit() -> Callable

shutdown()



shutdownNow()

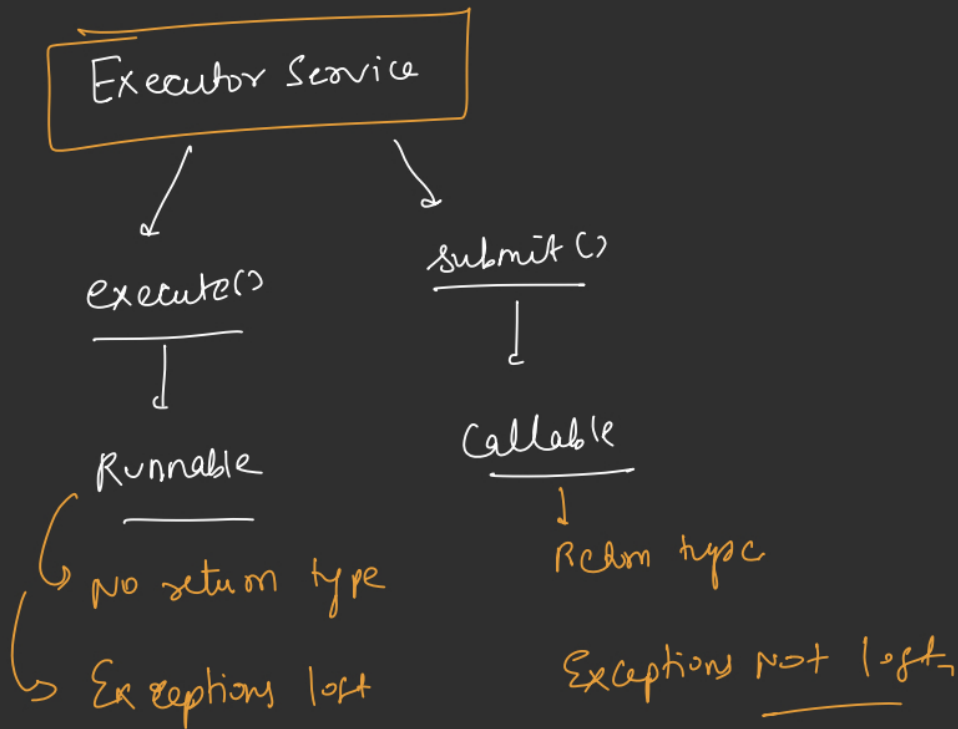
↳ guarantee



List<Callable> tasks

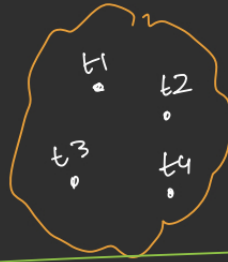
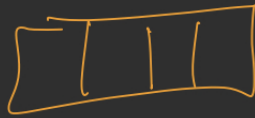
`List<Callable> tasks = List.of(
 () -> 10,
 () -> 20,
 () -> 30
);`

`List<Future> futures`
 ↑
executor.invokeAll(tasks);



Thread Pool Executor

- ↳ Configurable engine
- ↳ Worker thread.
- ↳ queue to hold extra tasks.
- ↳ Rules



ExecutorService executor =

Executors.newFixedThreadPool(2);

new ThreadPoolExecutor(2);

ThreadPoolExecutor (

int curPoolSize,

int maximumPoolSize,

long keepAliveTime,

TimeUnit unit,

BlockingQueue<Runnable> workQueue

);

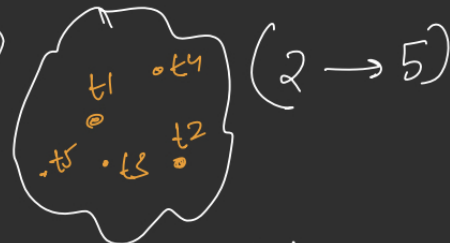
curPoolSize : (2)

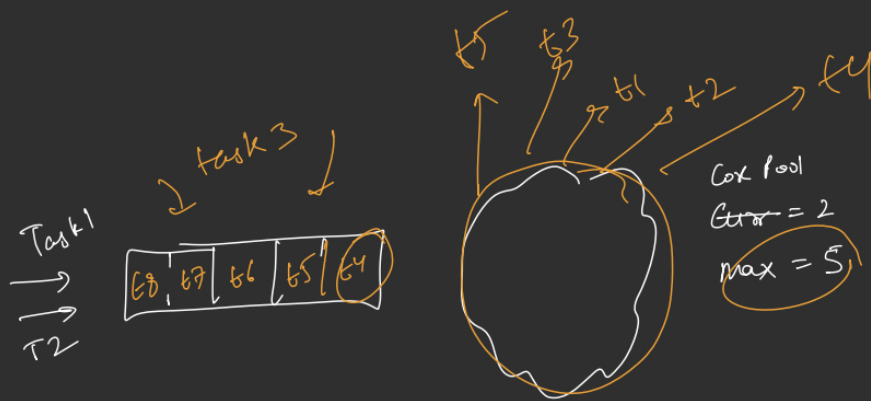
maxPoolSize : (5)

workQueue : Fixed / unlimited

keepAliveTime : (10)

TimeUnit (seconds)



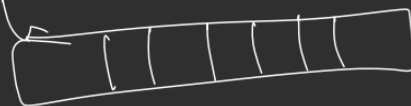


Rule:

- ① if $\text{currentThread} < \text{CorePoolSize}$
create a thread → Task execution
- ② else if Queue has space →
put task in Q
- ③ else $\text{threads} < \text{max}$
- ④ else → reject task

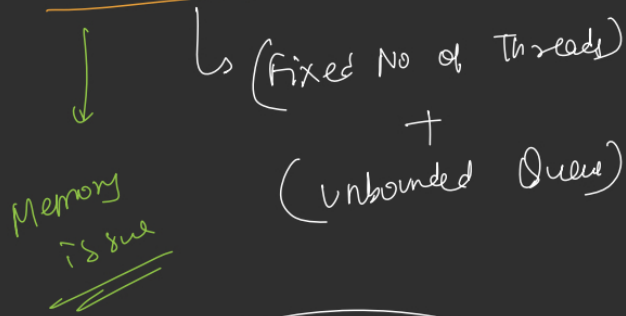
Types of Queue

- ↳ ArrayBlockingQueue
↳ (5)
- ↳ LinkedBlockingQueue



Executors • `newFixedThreadPool()`
• `newCachedThreadPool()`

Fixed Thread Pool



Cached Thread Pool



(Unlimited Threads)
+
(No Queue)

Single Thread Pool

↳ (1 thread)
+
(Queue)

Serializable

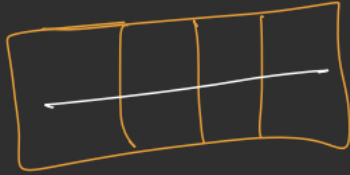
Scheduled Thread Pool

↳ Run task in future.

```
scheduler.schedule(() -> {  
    System.out.println("Hello");
```

```
}, 2, TimeUnit.SECONDS);
```

```
scheduler.scheduleAtFixedRate(  
    () -> {}, 0, 2, TimeUnit.SECONDS);
```



Full



Max

Rejection Policy

↳ ① Abort Policy

↳ Rejected Execution

↳ ② Discard Policy

↳ silently discard

↳ ③ Discard Oldest Policy

